

CONHECIMENTO E RACIOCÍNIO

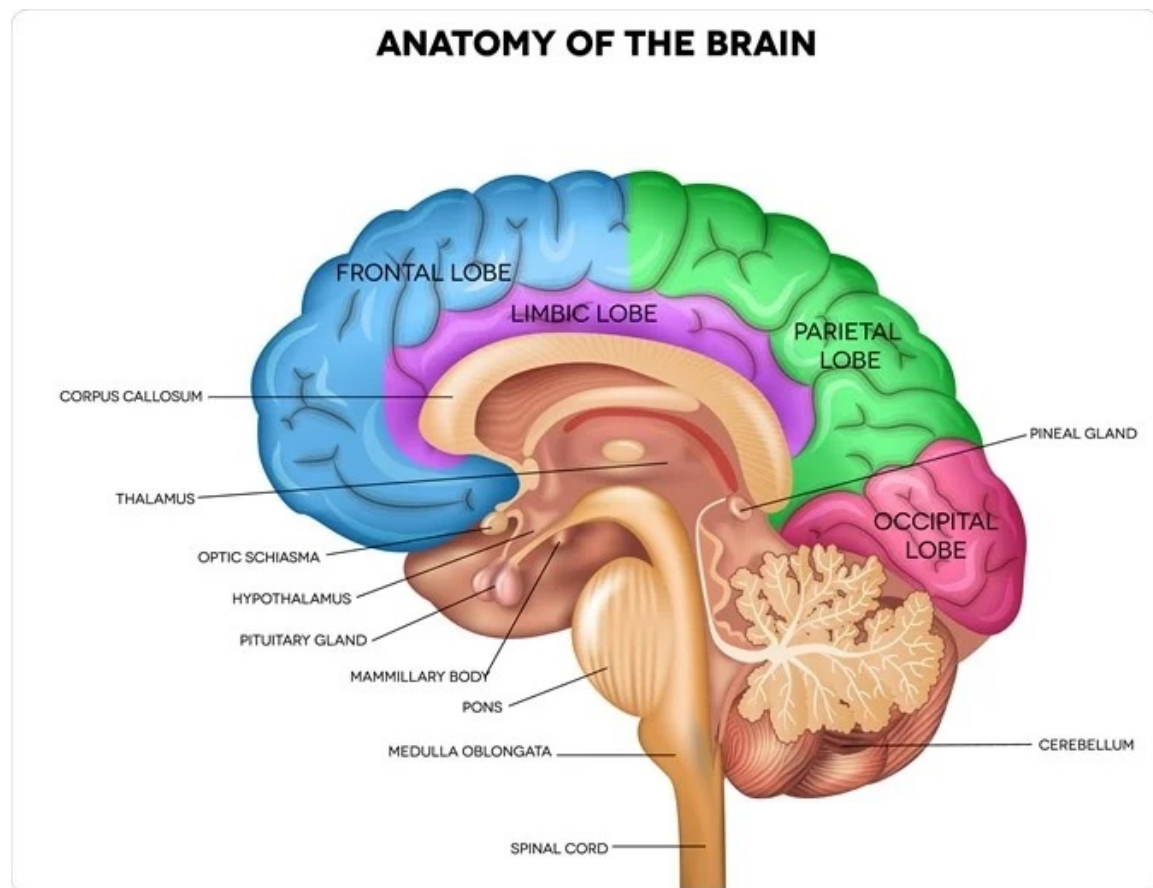
8 - Redes Neurais #1



2023/24

Redes Neurais: inspiração biológica

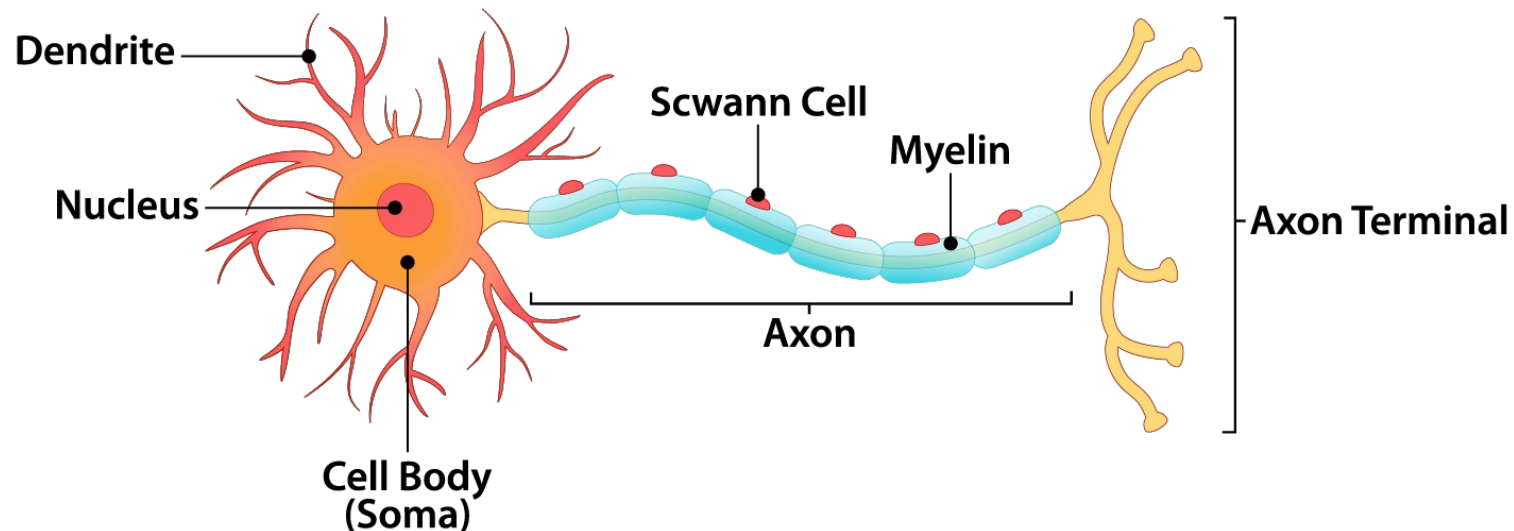
- Como aprendem os humanos?
 - Cérebro como inspiração



Redes Neurais: inspiração biológica

- Cérebro: enorme complexidade
 - Unidade mais simples: neurónio

NEURON ANATOMY



Redes Neurais: inspiração biológica

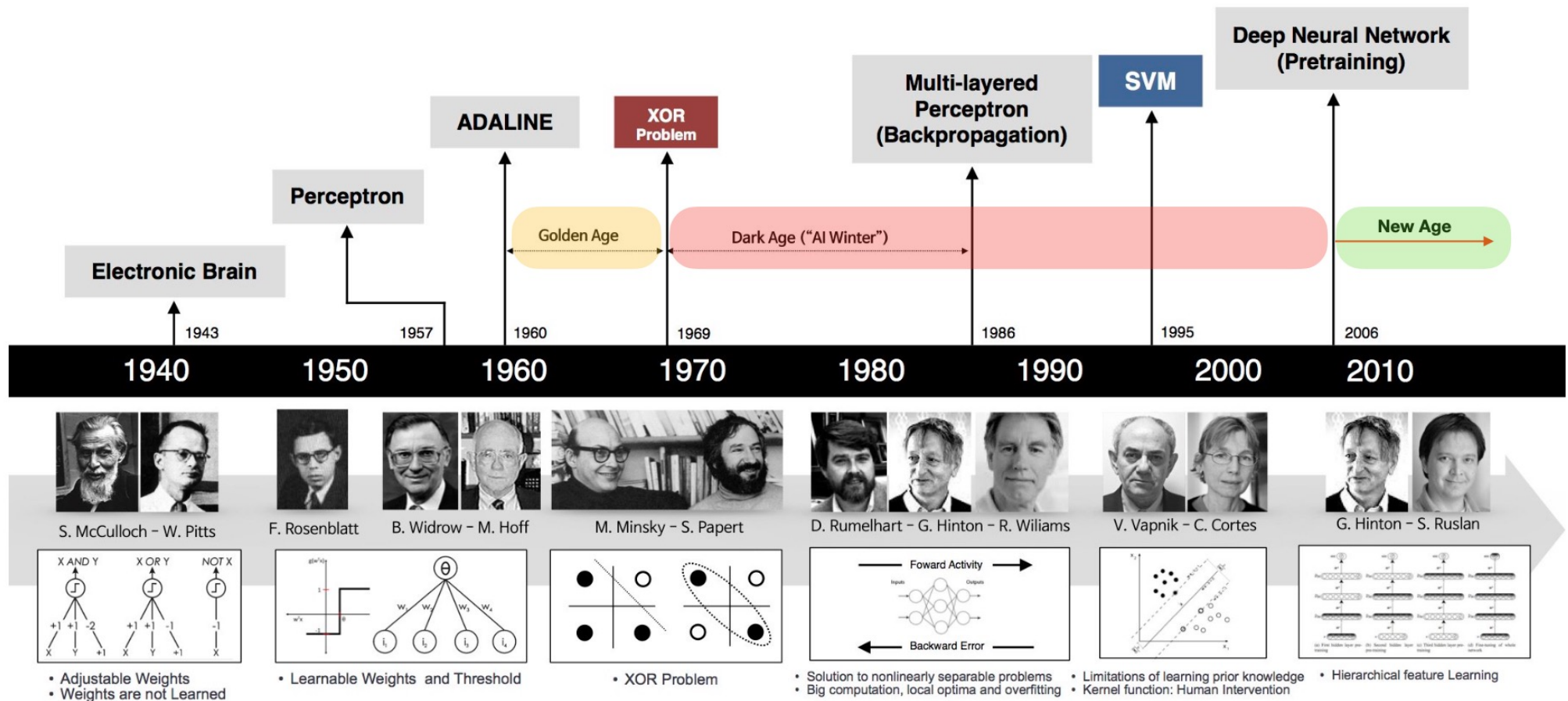
- As **redes neurais artificiais** foram inspiradas no funcionamento do cérebro humano, nomeadamente na estrutura e funcionamento dos neurónios
- As primeiras versões destas redes foram propostas nos anos 40 e resolviam problemas muito simples
- Atualmente existem inúmeras versões de redes neurais artificiais capazes de resolver problemas muito complexos

DEEP LEARNING

Redes Neurais: inspiração biológica

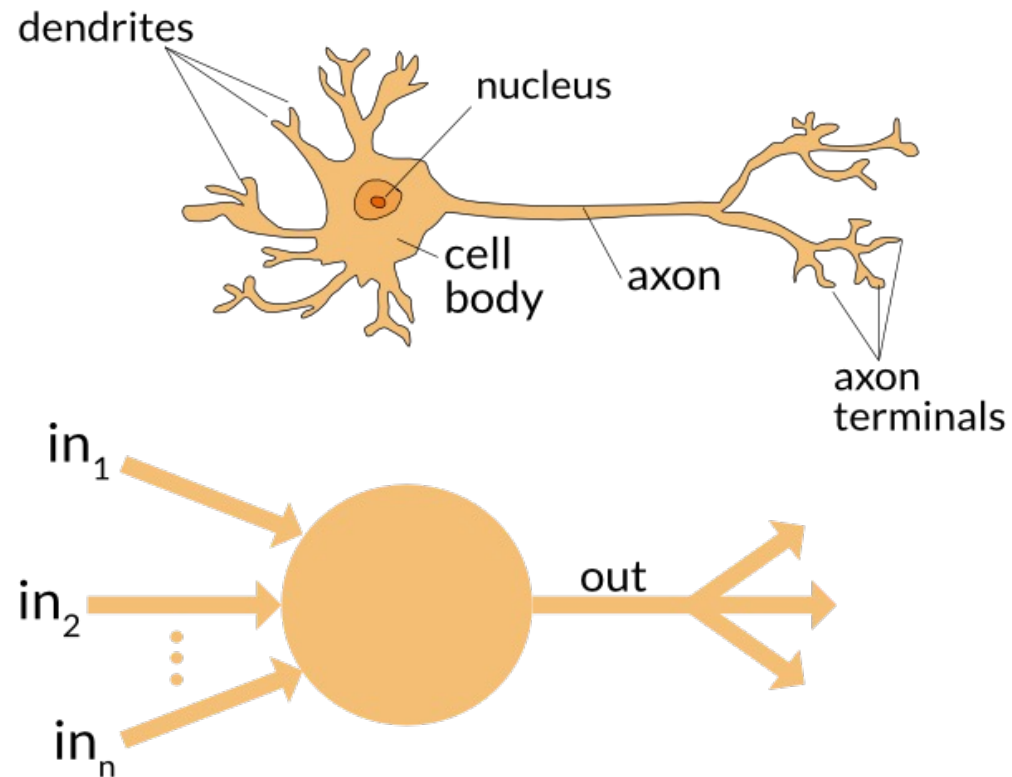
- Como aprendem as máquinas?
 - Como tudo começou:
 - Neurónio artificial - **McCulloch-Pitts** (1943)
 - Abordagens connexionistas
 - Redes neurais de grande complexidade
 - Ligação entre diferentes unidades simples geram redes muito poderosas:
 - Quando um neurónio “dispara”, gera um sinal que viaja para outros neurónios que, por sua vez, aumentam ou diminuem os seus estímulos de acordo com esse sinal.

Redes Neuronais: um pouco de história



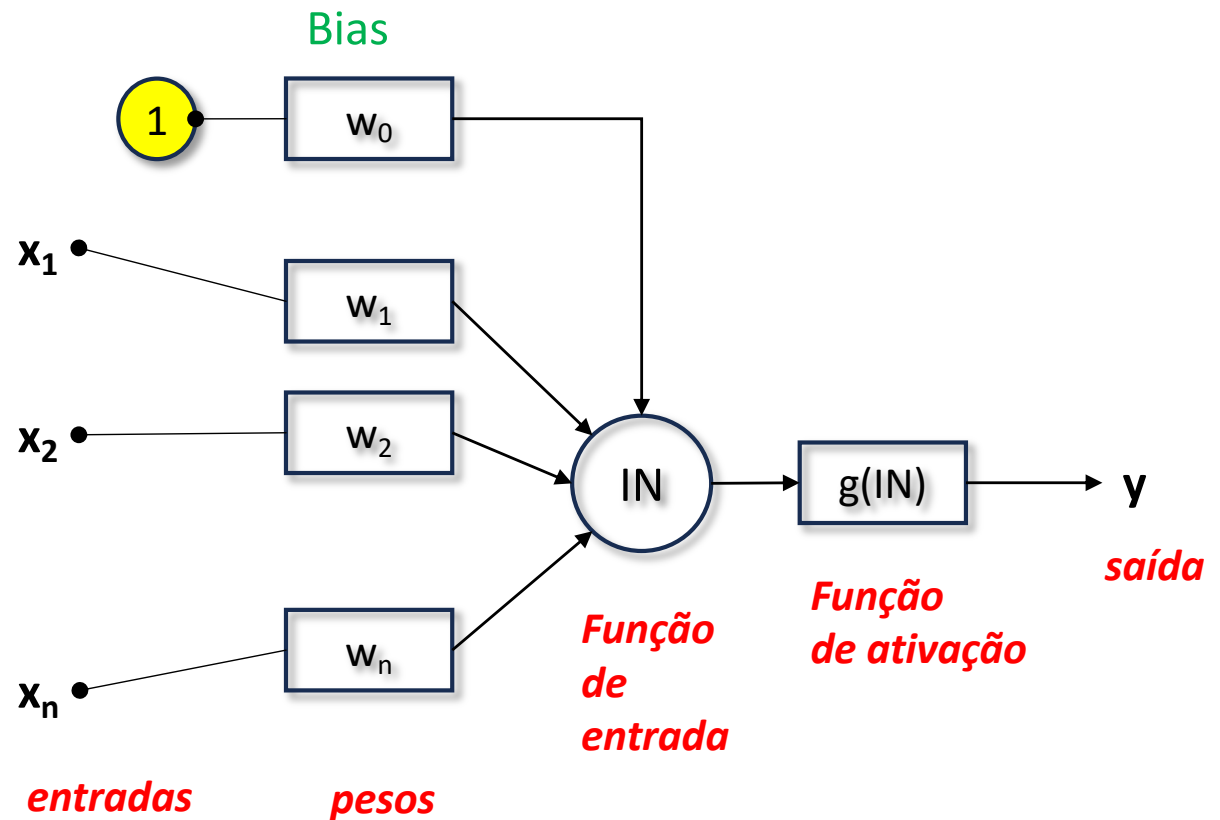
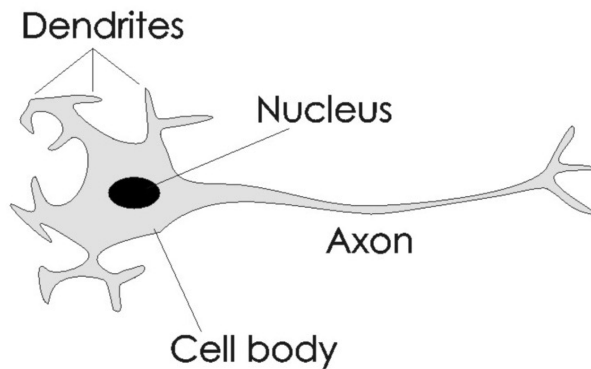
Redes Neurais: McCulloch-Pitts

- Natural *versus* Artificial

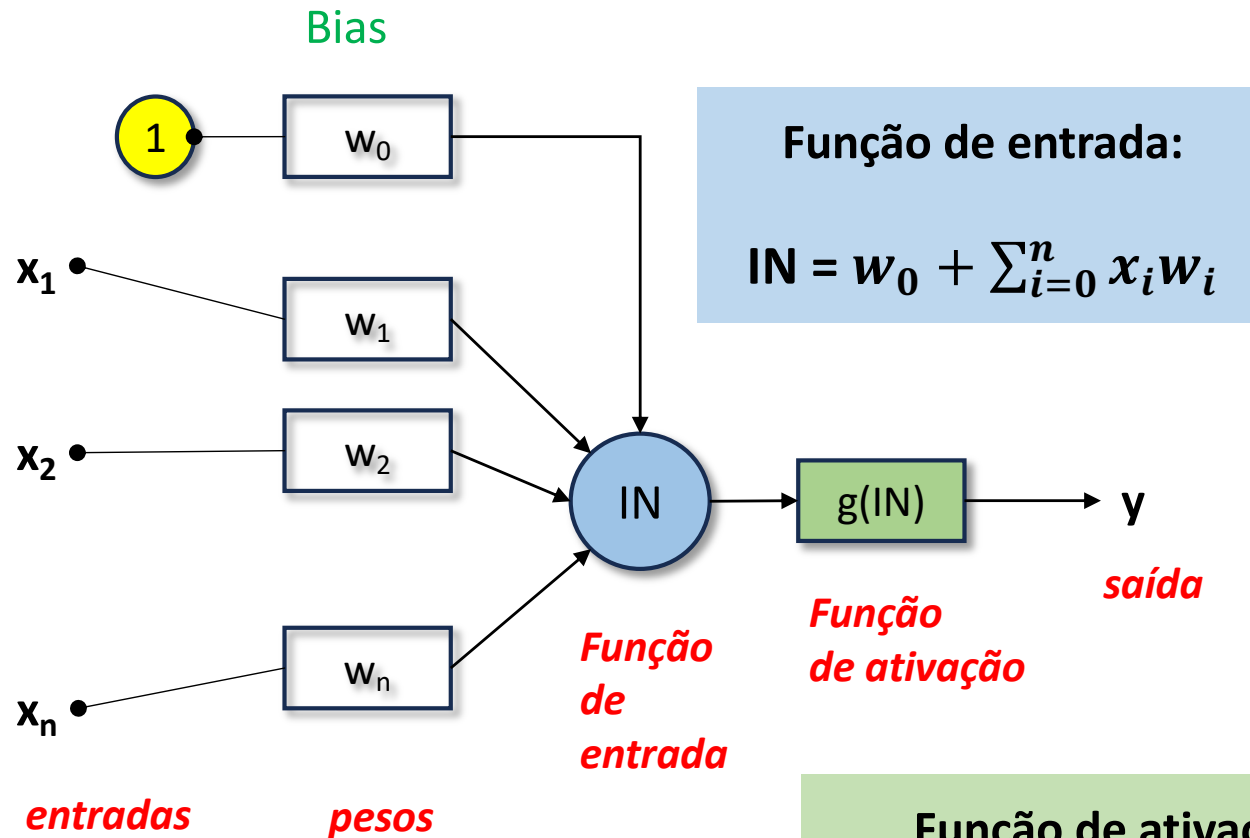


Redes Neurais: neurónio artificial

- Natural *versus* Artificial



Redes Neurais: neurónio artificial



Redes Neurais Artificiais

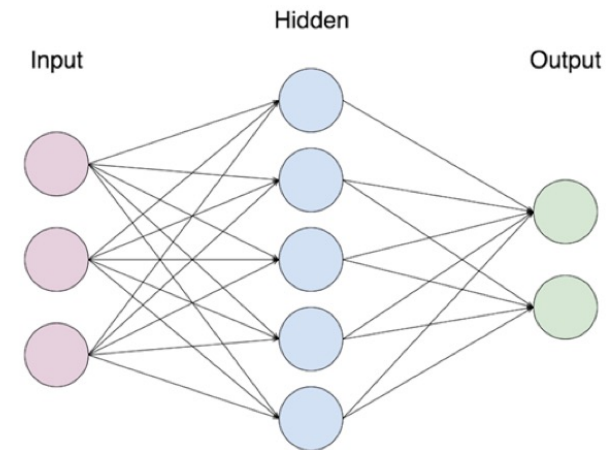
- **Topologia**
- **Função de ativação**
- **Algoritmo de aprendizagem**

Redes Neuronais Artificiais

Topologia

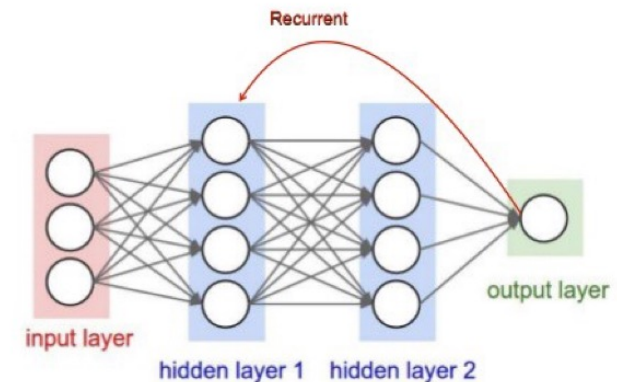
Número de camadas / neurónios

- Camada de entrada – variáveis de entrada
- Camada de saída – variáveis de saída
- Camadas escondidas:
 - quantas?
 - quantos neurónios?



Ligação dos neurónios

- Todos ligados?
- Direção das ligações



Redes Neuronais Artificiais

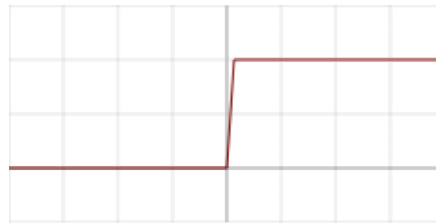
Função de ativação

Identity Function



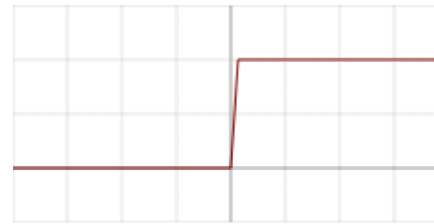
$$y = x$$

Binary Step



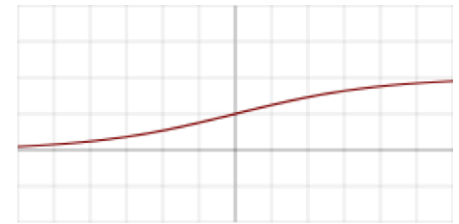
$$y = \begin{cases} 1 & \text{if } x \geq 0 \\ 0 & \text{if } x < 0 \end{cases}$$

Sign Function



$$y = \begin{cases} 1 & \text{if } x \geq 0 \\ -1 & \text{if } x < 0 \end{cases}$$

Logistic Function
or Sigmoid



$$y = \frac{1}{1 + e^{-x}}$$

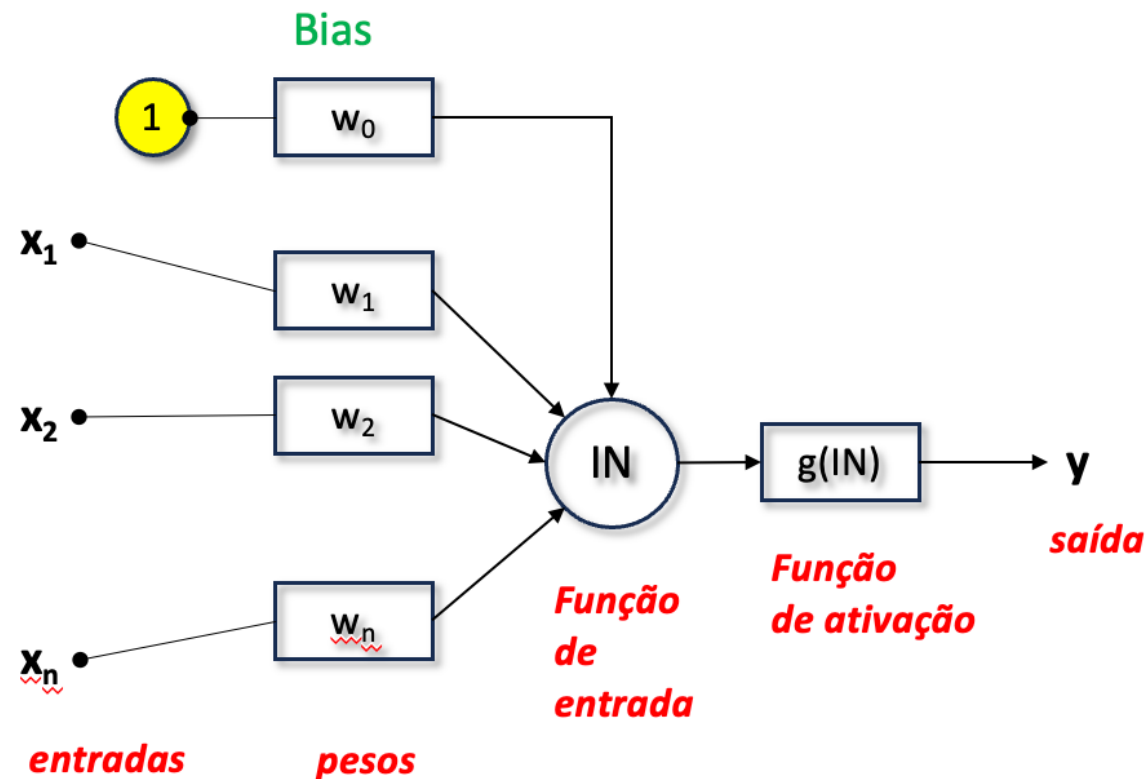
$$y \in [0,1]$$

Existem muitas outras!!

Redes Neuronais Artificiais

Algoritmo de aprendizagem

Aprendizagem: ajustar o valor dos pesos w_i



Redes Neurais: Regra de Hebb

- Proposta pelo neurocientista Donald Hebb em 1949 (*The Organization of Behavior*).

- **Aprendizagem natural**

*“when the axon of a neuron A is close enough to excite a neuron B, and takes part on its activation in a **repetitive and persistent way**, some type of growth process or metabolic change takes place in one or both cells , so that increases the efficiency of cell A in activation of B”*



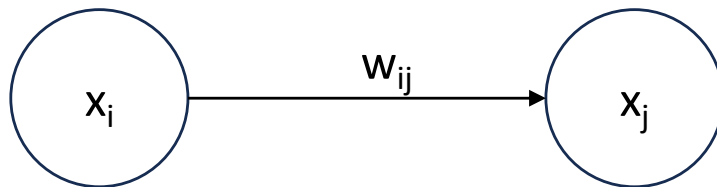
Aprendizagem artificial = modificação dos pesos w_i

Redes Neuronais Artificiais

Algoritmo de aprendizagem

Regra de Hebb:

se dois neurónios ativam simultaneamente, a sua ligação deve ser reforçada



$$w_{ij} = w_{ij} + \Delta w_{ij} \quad \text{Ajustar os pesos apenas se } x_i \text{ e } x_j \text{ são ativados (igual a 1)}$$

Redes Neurais Artificiais

Algoritmo de aprendizagem

Regra Delta (ou Widrow-Hoff):

- atualizar os pesos usando o **valor do erro**
- baseada no algoritmo do **gradiente descendente**

$$w_i = w_i + \Delta w_i$$

$$\Delta w_i = \eta * (\text{target}_i - y_i) * x_i$$

Redes Neuronais

Arquiteturas Básicas

- McCulloch-Pitts
- Perceptron / Adaline

O primeiro neurónio artificial

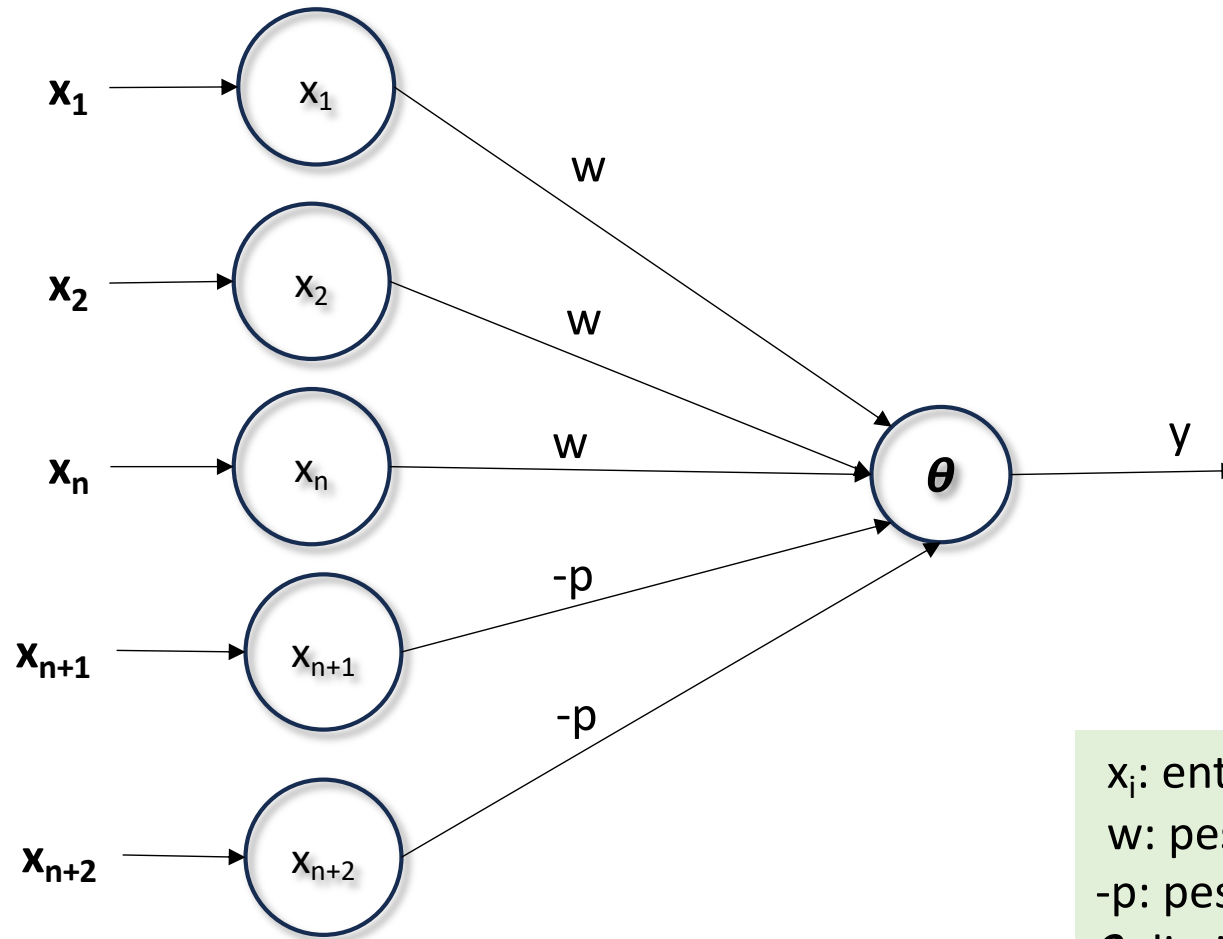
McCulloch-Pitts

1943

Redes Neurais: McCulloch-Pitts

- O primeiro modelo computacional de um neurónio (1943)
- Proposto por Warren McCulloch (neurocientista) e Walter Pitts (matemático)
- Também conhecido por **neurónio M-P**
- Modelo **binário**, valor de saída 1 ou 0
- Pesos podem ser de ativação (positivos) ou de inibição (negativos)
- **Não tem aprendizagem: pesos não são atualizados**
- Tem um limiar θ fixo que controla a função de ativação (saída)

Redes Neurais: McCulloch-Pitts



x_i : entradas
 w : pesos de ativação
 $-p$: pesos de inibição
 θ : limiar de ativação
 y : saída

Redes Neurais: McCulloch-Pitts

- A saída do neurónio M-P é controlada por um limiar de ativação fixo θ

Função de ativação

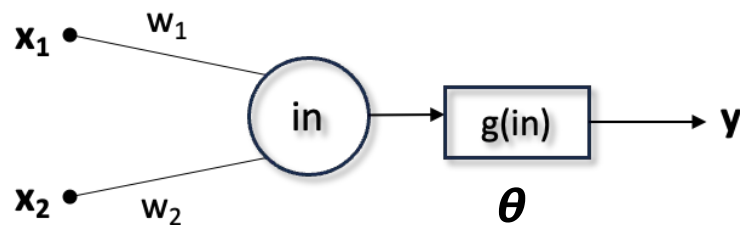
$$y = f(in) = \begin{cases} 1, y \geq \theta \\ 0, y < \theta \end{cases}$$

Com:

$$\theta \geq nw - p$$

Redes Neurais: McCulloch-Pitts

- Exemplo: usar o **M-P** para implementar a porta **AND**

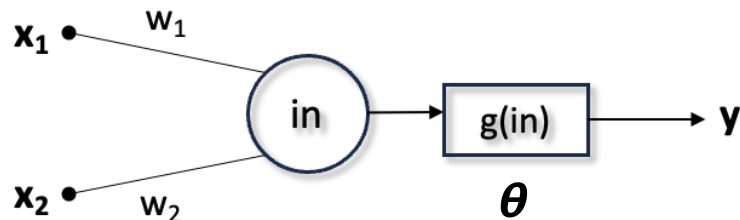


x_1	x_2	y
0	0	0
0	1	0
1	0	0
1	1	1

- o **M-P** não tem algoritmo de treino, i.e, não aprende
- Como atingir o objetivo?**
 - Analisar o problema e atribuir valores aos pesos

Redes Neurais: McCulloch-Pitts

Exemplo 1: usar o M-P para implementar a porta **AND**



x_1	x_2	y
0	0	0
0	1	0
1	0	0
1	1	1

Caso 1: $w_1 = 1$; $w_2 = 1$

$$\text{in} = \sum_i x_i w_i$$

$$\begin{aligned} (0, 0) &\rightarrow 0 * 1 + 0 * 1 = 0 \\ (0, 1) &\rightarrow 0 * 1 + 1 * 1 = 1 \\ (1, 0) &\rightarrow 1 * 1 + 0 * 1 = 1 \\ (1, 1) &\rightarrow 1 * 1 + 1 * 1 = 2 \end{aligned}$$

Caso 2: $w_1 = 1$; $w_2 = -1$

$$(0, -1, 1, 0)$$

Caso 3: $w_1 = -1$; $w_2 = 1$

$$(0, 1, -1, 0)$$

Caso 4: $w_1 = -1$; $w_2 = -1$

$$(0, -1, -1, -2)$$

$$\theta \geq nw - p$$

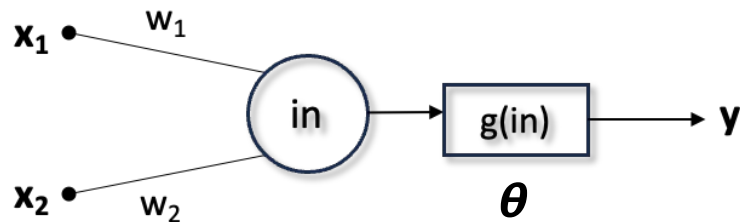
Só no caso 1 é possível separar as classes, com $\theta \geq 2$

$$y = f(\text{in}) = \begin{cases} 1, y \geq 2 \\ 0, y < 2 \end{cases}$$

$$\begin{aligned} (0, 0) &\rightarrow y = 0 \\ (0, 1) &\rightarrow y = 0 \\ (1, 0) &\rightarrow y = 0 \\ (1, 1) &\rightarrow y = 1 \end{aligned}$$

Redes Neurais: McCulloch-Pitts

Exemplo 2: usar o M-P para implementar a porta **OR**



x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	1

Caso 1: $w_1 = 1; w_2 = 1$

$$\text{in} = \sum_i x_i w_i$$

$$(0, 0) \rightarrow 0 * 1 + 0 * 1 = 0$$

$$(0, 1) \rightarrow 0 * 1 + 1 * 1 = 1$$

$$(1, 0) \rightarrow 1 * 1 + 0 * 1 = 1$$

$$(1, 1) \rightarrow 1 * 1 + 1 * 1 = 2$$

Caso 2: $w_1 = 1; w_2 = -1$

$$(0, -1, 1, 0)$$

Caso 3: $w_1 = -1; w_2 = 1$

$$(0, 1, -1, 0)$$

Caso 4: $w_1 = -1; w_2 = -1$

$$(0, -1, -1, -2)$$

Só no caso 1 é possível separar as classes, com $\theta \geq 1$

$$y = f(\text{in}) = \begin{cases} 1, & y \geq 1 \\ 0, & y < 1 \end{cases}$$

$$(0, 0) \rightarrow y = 0$$

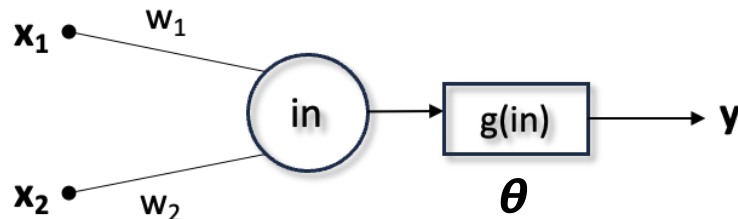
$$(0, 1) \rightarrow y = 1$$

$$(1, 0) \rightarrow y = 1$$

$$(1, 1) \rightarrow y = 1$$

Redes Neurais: McCulloch-Pitts

Exemplo 3: usar o M-P para implementar a porta **NOR**



x_1	x_2	y
0	0	1
0	1	0
1	0	0
1	1	0

Caso 1: $w_1 = 1$; $w_2 = 1$

$$\text{in} = \sum_i x_i w_i$$

$$(0, 0) \rightarrow 0 * 1 + 0 * 1 = 0$$

$$(0, 1) \rightarrow 0 * 1 + 1 * 1 = 1$$

$$(1, 0) \rightarrow 1 * 1 + 0 * 1 = 1$$

$$(1, 1) \rightarrow 1 * 1 + 1 * 1 = 2$$

Caso 2: $w_1 = 1$; $w_2 = -1$

$$(0, -1, 1, 0)$$

Caso 3: $w_1 = -1$; $w_2 = 1$

$$(0, 1, -1, 0)$$

Caso 4: $w_1 = -1$; $w_2 = -1$

$$(0, -1, -1, -2)$$

Só no caso 4 é possível separar as classes, com $\theta \geq 0$

$$y = f(\text{in}) = \begin{cases} 1, & y \geq 0 \\ 0, & y < 0 \end{cases}$$

$$(0, 0) \rightarrow y = 1$$

$$(0, 1) \rightarrow y = 0$$

$$(1, 0) \rightarrow y = 0$$

$$(1, 1) \rightarrow y = 0$$

Neurónio artificial com aprendizagem

Perceptron

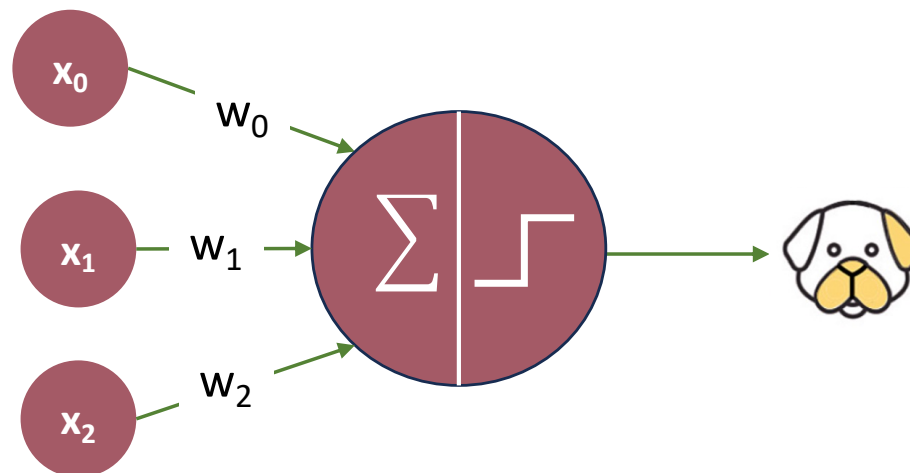
1957

ADALINE: Adaptive Linear Neuron

1960

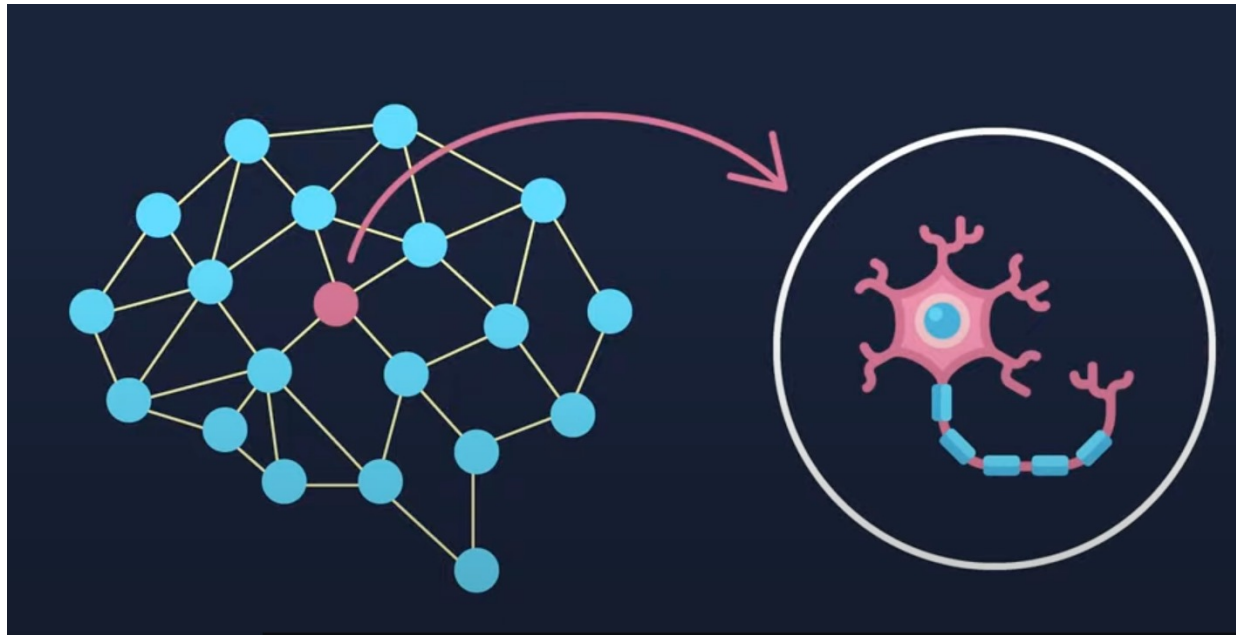
Redes Neurais: Perceptron

- Frank Rosenblatt cria o **perceptron** (1957)
- Widrow e Hoff (1960) propõem o **ADALINE**
 - igual ao perceptron mas com um algoritmo de treino diferente
- Unidade mais simples de uma rede neuronal
- Saída de um neurónio é binária



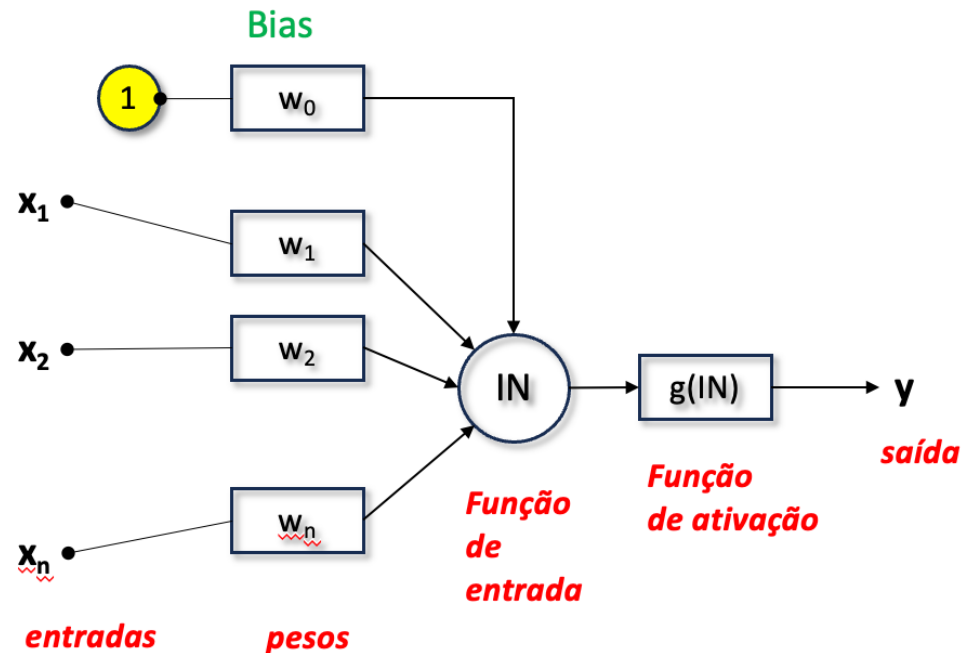
Redes Neurais: Perceptron

- O **perceptron** é a unidade mais simples de uma rede neuronal
- A ligação de vários perceptrons forma uma rede neuronal artificial

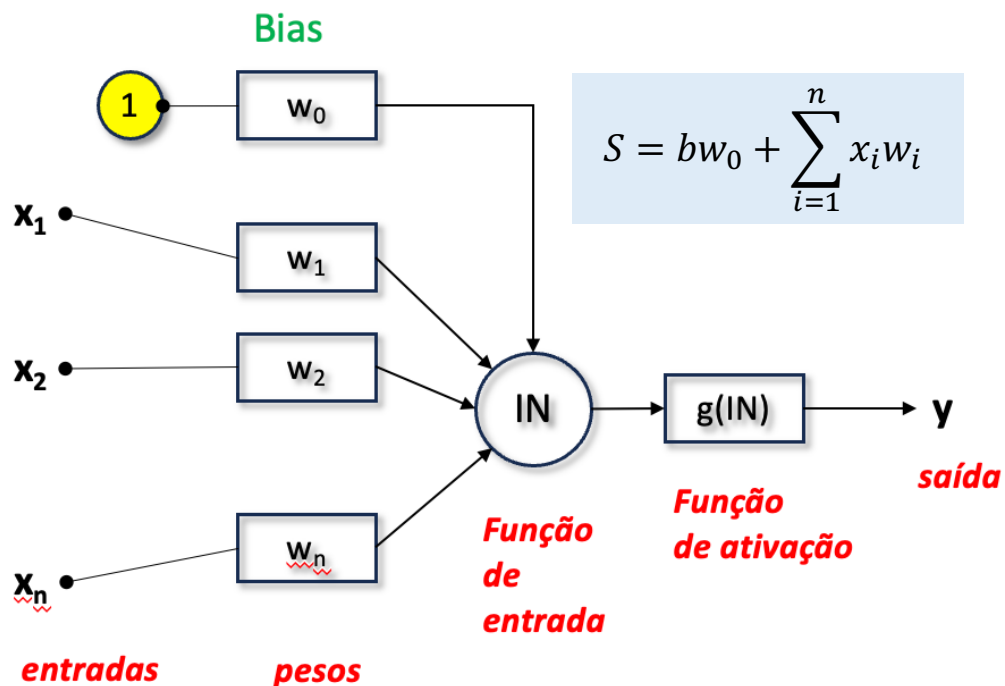


Redes Neurais: Perceptron

- Com o perceptron foi introduzido o **algoritmo de treino!!**
 - **Aprender** os valores corretos para os pesos w_i
 - Aprendizagem supervisionada: aprende a relação entre as entradas e a saída desejada (*target*)



Redes Neuronais: Perceptron



Perceptron

$$\Delta w_i = \eta * \text{target} * x_i$$

Função de ativação Step (degrau)

$$y = g(in) = \begin{cases} 1, & \text{if } S \geq 0 \\ 0, & \text{if } S < 0 \end{cases}$$

Aprendizagem supervisionada:

Iterativamente ajusta o valor dos pesos usando o erro

$$W_i = W_i + \Delta w_i$$

REGRA DELTA

ADALINE

$$\Delta w_i = \eta * (\text{target} - y) * x_i$$

↑
Coeficiente de aprendizagem

Redes Neurais: Perceptron

Algoritmo:

1. Pesos $\mathbf{w}_i$ iniciais: aleatórios
2. Iniciar com um conjunto de dados supervisionados

Repetir

3. Para cada entrada $\mathbf{x}_i$, calcular a saída $\mathbf{y}$ do perceptron
 - Calcular o erro: saída desejada – saída obtida $\mathbf{y}$
 - Usar o erro para ajustar os pesos: regra delta

Passos 3 é repetido até o erro ser zero (ou outro critério de paragem)

Redes Neurais: perceptron

Aprendizagem supervisionada

Conjunto de dados supervisionado

para cada entrada é conhecida a saída desejada (target)

Exemplo	X1	X2	Target
1	0	0	0
2	0	1	0
3	1	0	0
4	1	1	1

Redes Neurais: perceptron

- **Conceitos importantes**

- **Época**

- número de vezes que o algoritmo processa o dataset

- **Número de exemplos**

- quantos exemplos do dataset são processados antes do modelo ajustar os pesos

- **Iterações**

- número de vezes que o algoritmo processa cada exemplo até completar uma época

Redes Neurais: perceptron

treino: Como calcular o erro e ajustar os pesos?

REGRA DELTA

1. Calcular o erro:

$$\text{erro} = \text{target} - y$$

2. Ajustar os pesos

$$\Delta w_i = \eta * (\text{target} - y) * x_i$$

$$w_i = w_i + \Delta w_i$$

η : coeficiente de aprendizagem

- **PERCEPTRON**

EXEMPLO

Aprender a porta **OR**

Redes Neurais: perceptron

exemplo - treino

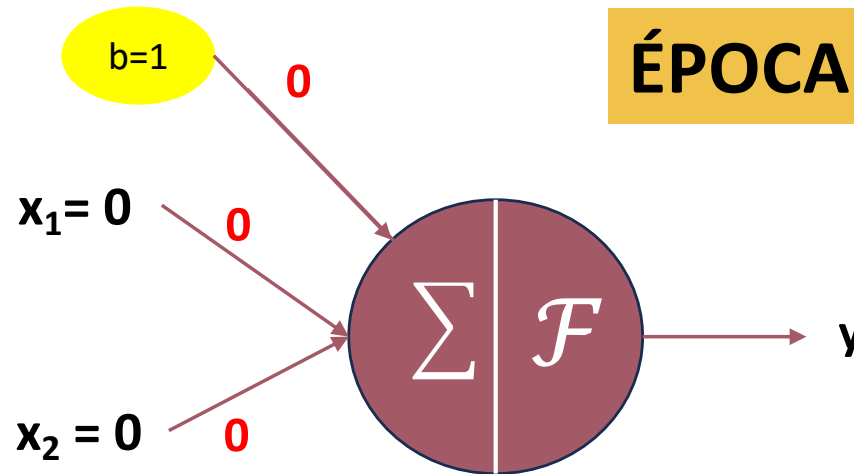
Exemplo	X1	X2	Target
1	0	0	0
2	0	1	1
3	1	0	1
4	1	1	1

$$S = 1 * 0 + 0 * 0 + 0 * 0 = 0$$

$$S = bw_0 + \sum_{i=1}^n x_i w_i$$

Função de ativação Step (Degrau)

$$y = \mathcal{F}(S) = \begin{cases} 1, & \text{if } S \geq 0 \\ 0, & \text{if } S < 0 \end{cases}$$



ÉPOCA 1 – exemplo1

$$\eta = 0.5$$

REGRA DELTA

$$w_0 = w_0 + \eta * (\text{target} - y) * b$$

$$w_1 = w_1 + \eta * (\text{target} - y) * x_1$$

$$w_2 = w_2 + \eta * (\text{target} - y) * x_2$$

$$y = 1 \quad \text{Target desejado} = 0$$

ERRO = -1

$$w_0 = 0 + 0.5 * (0 - 1) * 1 = -0.5$$

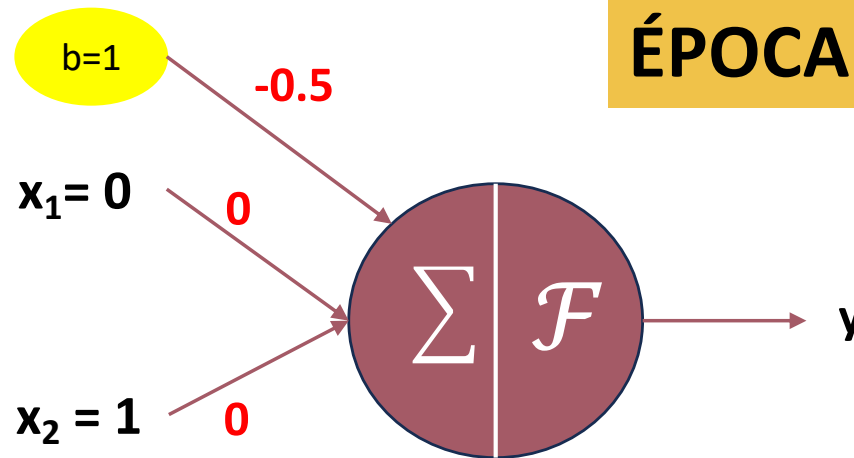
$$w_1 = 0 + 0.5 * (0 - 1) * 0 = 0$$

$$w_2 = 0 + 0.5 * (0 - 1) * 0 = 0$$

Redes Neurais: perceptron

exemplo - treino

Exemplo	X1	X2	Target
1	0	0	0
2	0	1	1
3	1	0	1
4	1	1	1



ÉPOCA 1 – exemplo2

$$S = 1 * (-0.5) + 0 * 0 + 0 * 1 = -0.5$$

$$S = bw_0 + \sum_{i=1}^n x_i w_i$$

$$y = 0$$

Target desejado = 1

ERRO = 1

Função de ativação Step (Degrau)

$$y = \mathcal{F}(S) = \begin{cases} 1, & \text{if } S \geq 0 \\ 0, & \text{if } S < 0 \end{cases}$$

$$\eta = 0.5$$

REGRA DELTA

$$w_0 = w_0 + \eta * (\text{target} - y) * b$$

$$w_1 = w_1 + \eta * (\text{target} - y) * x_1$$

$$w_2 = w_2 + \eta * (\text{target} - y) * x_2$$

$$w_0 = -0.5 + 0.5 * (1 - 0) * 1 = 0$$

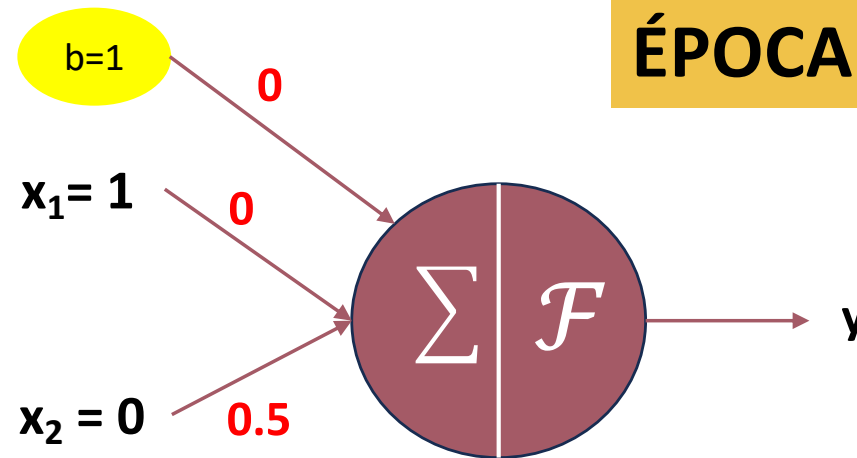
$$w_1 = 0 + 0.5 * (1 - 0) * 0 = 0$$

$$w_2 = 0 + 0.5 * (1 - 0) * 1 = 0.5$$

Redes Neurais: perceptron

exemplo - treino

Exemplo	X1	X2	Target
1	0	0	0
2	0	1	1
3	1	0	1
4	1	1	1



$$S = 1 * 0 + 1 * 0 + 0 * 0.5 = 0$$

$$S = bw_0 + \sum_{i=1}^n x_i w_i$$

$$y = 1$$

Target desejado = 1

ERRO = 0

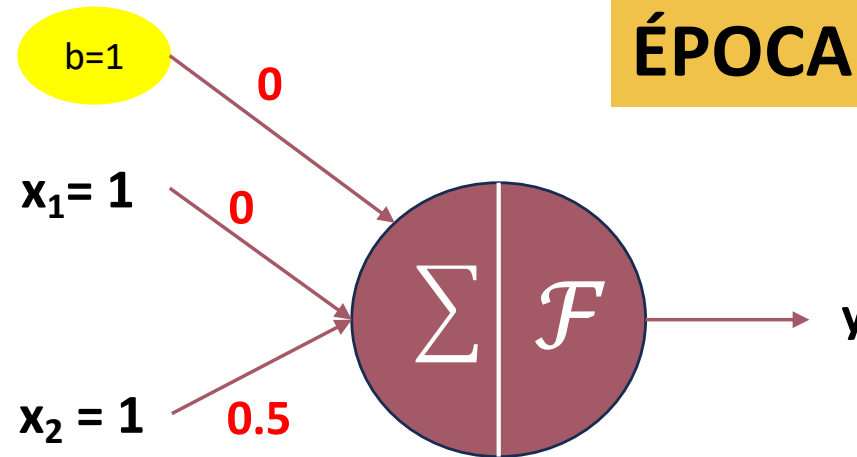
Função de ativação Step (Degrau)

$$y = \mathcal{F}(S) = \begin{cases} 1, & \text{if } S \geq 0 \\ 0, & \text{if } S < 0 \end{cases}$$

Redes Neurais: perceptron

exemplo - treino

Exemplo	X1	X2	Target
1	0	0	0
2	0	1	1
3	1	0	1
4	1	1	1



ÉPOCA 1 – exemplo4

$$S = 1 * 0 + 1 * 0 + 1 * 0.5 = 0.5$$

$$S = bw_0 + \sum_{i=1}^n x_i w_i$$

$$y = 1$$

Target desejado = 1

ERRO = 0

Função de ativação Step (Degrau)

$$y = \mathcal{F}(S) = \begin{cases} 1, & \text{if } S \geq 0 \\ 0, & \text{if } S < 0 \end{cases}$$

FIM ÉPOCA 1

Redes Neurais: perceptron

exemplo - treino

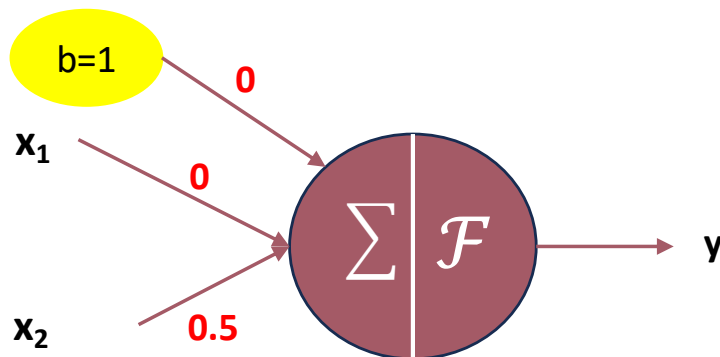
Exemplo	X1	X2	Target
1	0	0	0
2	0	1	1
3	1	0	1
4	1	1	1

Exemplo 1

$$S = 1 * 0 + 0 * 0 + 0 * 0.5 = 0$$

$$y = 1$$

erro = -1



$$\eta = 0.5$$

REGRA DELTA

$$w_0 = w_0 + \eta * (\text{target} - y) * b$$

$$w_1 = w_1 + \eta * (\text{target} - y) * x_1$$

$$w_2 = w_2 + \eta * (\text{target} - y) * x_2$$

$$w_0 = 0 + 0.5 * (0 - 1) * 1 = -0.5$$

$$w_1 = 0 + 0.5 * (0 - 1) * 0 = 0$$

$$w_2 = 0.5 + 0.5 * (0 - 1) * 0 = 0.5$$

ÉPOCA 2

$$S = bw_0 + \sum_{i=1}^n x_i w_i$$

Função de ativação Step (Degrau)

$$y = \mathcal{F}(S) = \begin{cases} 1, & \text{if } S \geq 0 \\ 0, & \text{if } S < 0 \end{cases}$$

Redes Neurais: perceptron

exemplo - treino

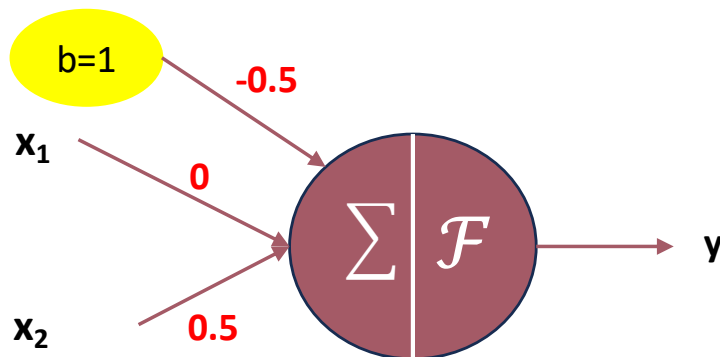
Exemplo	X1	X2	Target
1	0	0	0
2	0	1	1
3	1	0	1
4	1	1	1

Exemplo 2

$$S = 1 * (-0.5) + 0 * 0 + 1 * 0.5 = 0$$

$$y = 1$$

erro = 0



ÉPOCA 2

$$S = bw_0 + \sum_{i=1}^n x_i w_i$$

Função de ativação Step (Degrau)

$$y = \mathcal{F}(S) = \begin{cases} 1, & \text{if } S \geq 0 \\ 0, & \text{if } S < 0 \end{cases}$$

Redes Neurais: perceptron

exemplo - treino

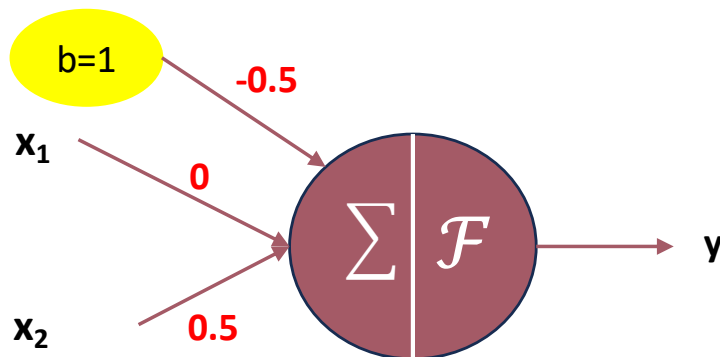
Exemplo	X1	X2	Target
1	0	0	0
2	0	1	1
3	1	0	1
4	1	1	1

Exemplo 3

$$S = 1 * (-0.5) + 1 * 0 + 0 * 0.5 = -0,5$$

$$y = 0$$

$$\text{erro} = 1$$



ÉPOCA 2

$$S = bw_0 + \sum_{i=1}^n x_i w_i$$

Função de ativação Step (Degrau)

$$y = \mathcal{F}(S) = \begin{cases} 1, & \text{if } S \geq 0 \\ 0, & \text{if } S < 0 \end{cases}$$

$$\eta = 0.5$$

REGRA DELTA

$$w_0 = w_0 + \eta * (\text{target} - y) * b$$

$$w_1 = w_1 + \eta * (\text{target} - y) * x_1$$

$$w_2 = w_2 + \eta * (\text{target} - y) * x_2$$

$$w_0 = -0.5 + 0.5 * (1 - 0) * 1 = 0$$

$$w_1 = 0 + 0.5 * (1 - 0) * 1 = 0.5$$

$$w_2 = 0.5 + 0.5 * (1 - 0) * 0 = 0.5$$

Redes Neurais: perceptron

exemplo - treino

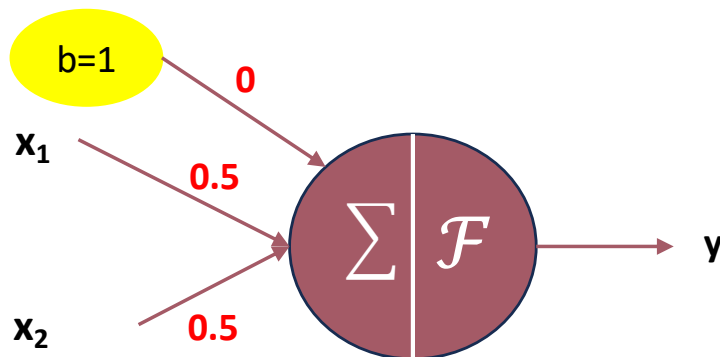
Exemplo	X1	X2	Target
1	0	0	0
2	0	1	1
3	1	0	1
4	1	1	1

Exemplo 4

$$S = 1 * 0 + 1 * 0.5 + 1 * 0.5 = 1$$

$$y = 1$$

erro = 0



ÉPOCA 2

$$S = bw_0 + \sum_{i=1}^n x_i w_i$$

Função de ativação Step (Degrau)

$$y = \mathcal{F}(S) = \begin{cases} 1, & \text{if } S \geq 0 \\ 0, & \text{if } S < 0 \end{cases}$$

FIM ÉPOCA 2

Redes Neurais: perceptron

exemplo - treino

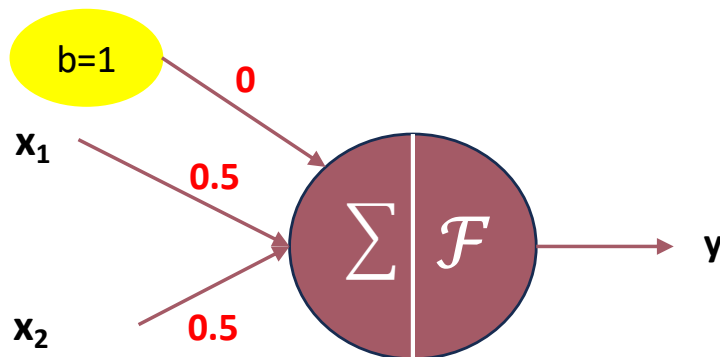
Exemplo	X1	X2	Target
1	0	0	0
2	0	1	1
3	1	0	1
4	1	1	1

Exemplo 1

$$S = 1 * 0 + 0 * 0.5 + 0 * 0.5 = 0$$

$$y = 1$$

erro = -1



ÉPOCA 3

$$S = bw_0 + \sum_{i=1}^n x_i w_i$$

Função de ativação Step (Degrau)

$$y = \mathcal{F}(S) = \begin{cases} 1, & \text{if } S \geq 0 \\ 0, & \text{if } S < 0 \end{cases}$$

$$\eta = 0.5$$

REGRA DELTA

$$w_0 = w_0 + \eta * (\text{target} - y) * b$$

$$w_1 = w_1 + \eta * (\text{target} - y) * x_1$$

$$w_2 = w_2 + \eta * (\text{target} - y) * x_2$$

$$w_0 = 0 + 0.5 * (0 - 1) * 1 = -0.5$$

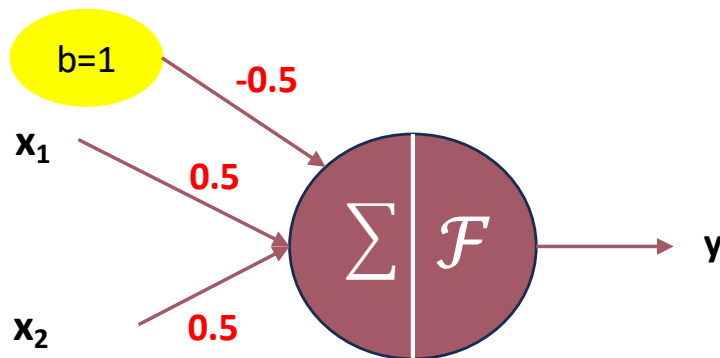
$$w_1 = 0.5 + 0.5 * (0 - 1) * 0 = 0.5$$

$$w_2 = 0.5 + 0.5 * (0 - 1) * 0 = 0.5$$

Redes Neurais: perceptron

exemplo - treino

Exemplo	X1	X2	Target
1	0	0	0
2	0	1	1
3	1	0	1
4	1	1	1



ÉPOCA 3

$$S = bw_0 + \sum_{i=1}^n x_i w_i$$

Função de ativação Step (Degrau)

$$y = \mathcal{F}(S) = \begin{cases} 1, & \text{if } S \geq 0 \\ 0, & \text{if } S < 0 \end{cases}$$

Exemplo 2

$$S = 1 * (-0.5) + 0 * 0.5 + 1 * 0.5 = 0$$

$$y = 1$$

erro = 0

Exemplo 4

$$S = 1 * (-0.5) + 1 * 0.5 + 1 * 0.5 = 0.5$$

$$y = 1$$

erro = 0

Exemplo 3

$$S = 1 * (-0.5) + 1 * 0.5 + 0 * 0.5 = 0$$

$$y = 1$$

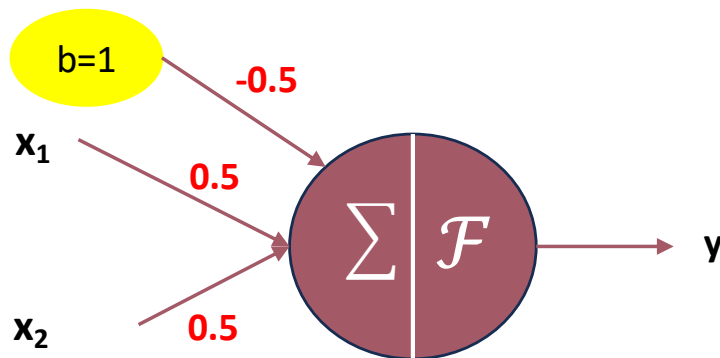
erro = 0

FIM ÉPOCA 3

Redes Neurais: perceptron

exemplo - treino

Exemplo	X1	X2	Target
1	0	0	0
2	0	1	1
3	1	0	1
4	1	1	1



ÉPOCA 4

$$S = bw_0 + \sum_{i=1}^n x_i w_i$$

Função de ativação Step (Degrau)

$$y = \mathcal{F}(S) = \begin{cases} 1, & \text{if } S \geq 0 \\ 0, & \text{if } S < 0 \end{cases}$$

Exemplo 1

$$S = 1 * (-0.5) + 0 * 0.5 + 0 * 0.5 = -0.5$$

$$y = 0$$

erro = 0

Exemplo 2

$$S = 1 * (-0.5) + 0 * 0.5 + 1 * 0.5 = 0$$

$$y = 1$$

erro = 0

Exemplo 3

$$S = 1 * (-0.5) + 1 * 0.5 + 0 * 0.5 = 0$$

$$y = 1$$

erro = 0

Exemplo 4

$$S = 1 * (-0.5) + 1 * 0.5 + 1 * 0.5 = 0.5$$

$$y = 1$$

erro = 0

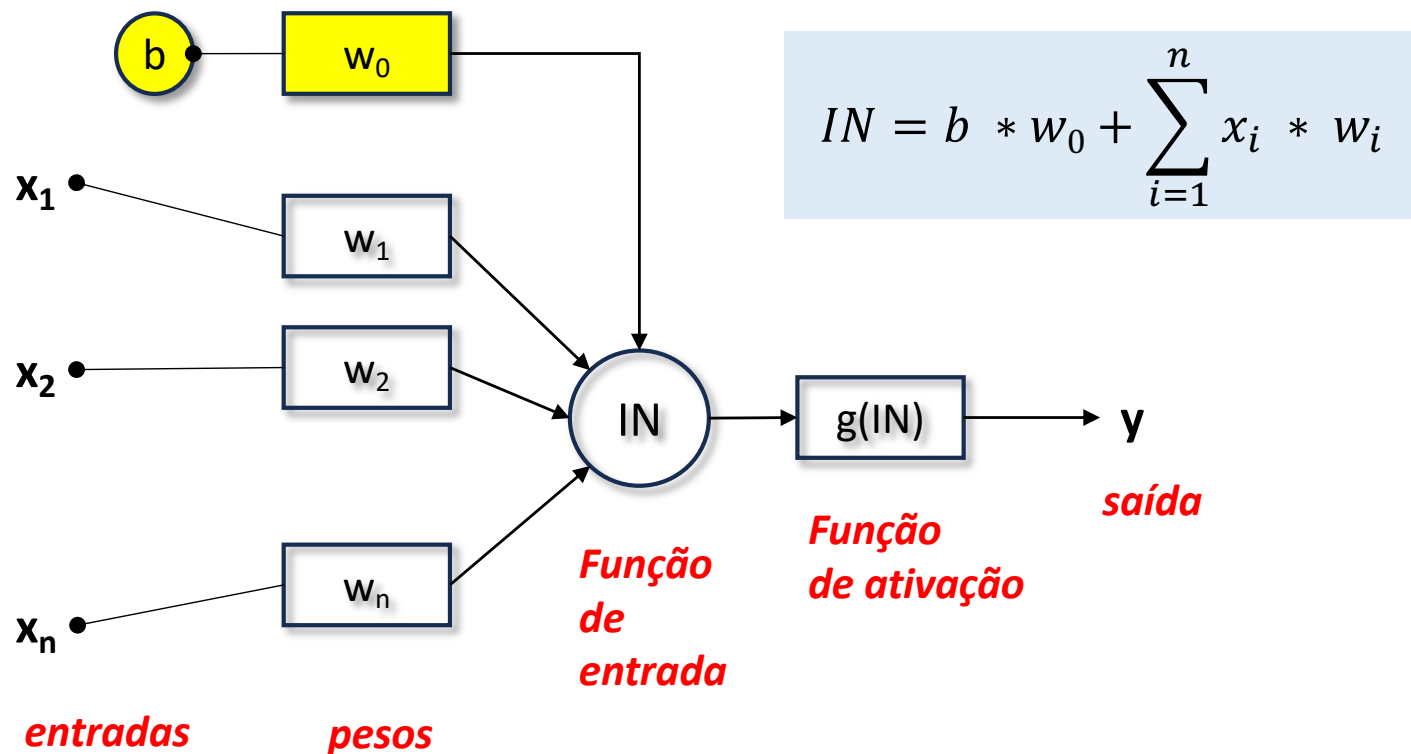
100% accuracy

FIM ÉPOCA 4

Redes Neurais: perceptron

Bias

- Bias: peso adicional que permite tornar o modelo mais eficiente



Redes Neurais: perceptron

Bias

- Ajuda a determinar o limiar (*threshold*) do disparo da função de ativação
- Torna o perceptron mais flexível
- O peso w_0 associado ao *bias* (b) também é ajustado com a regra delta

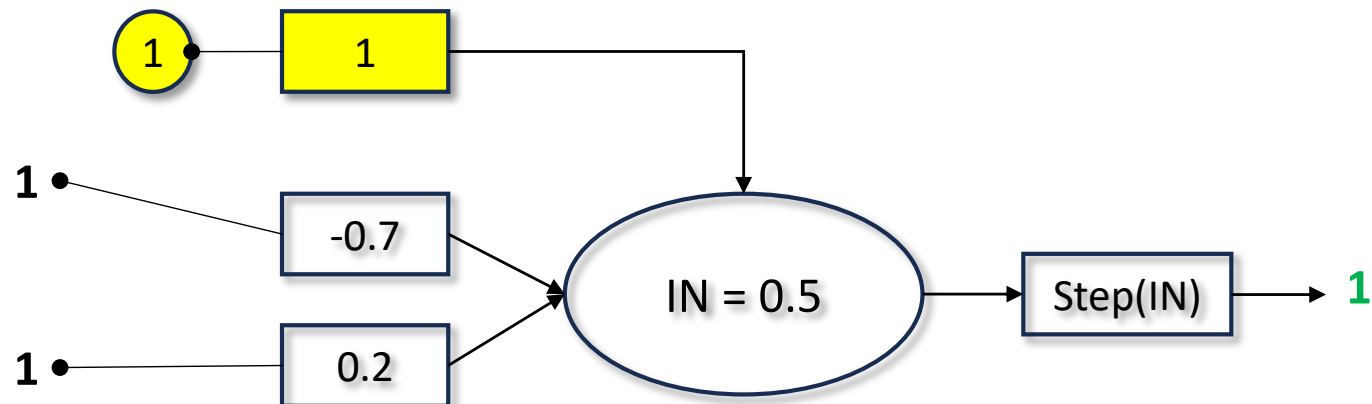
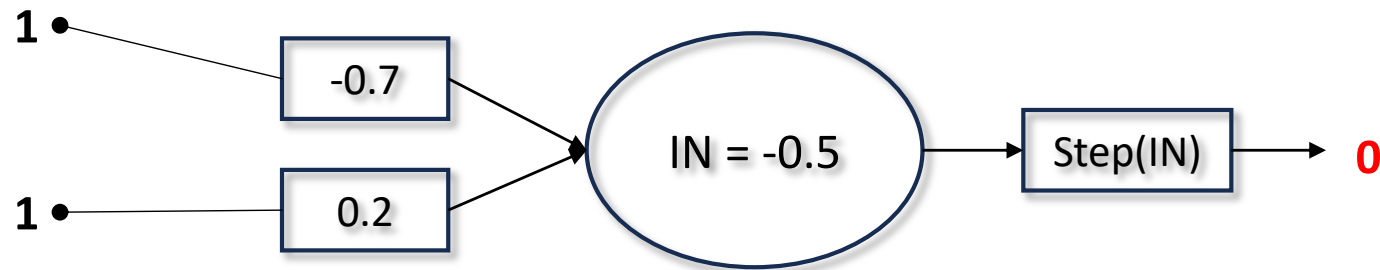
$$w_0 = w_0 + \eta * (\text{target} - y) * b$$

Redes Neurais: perceptron

Bias

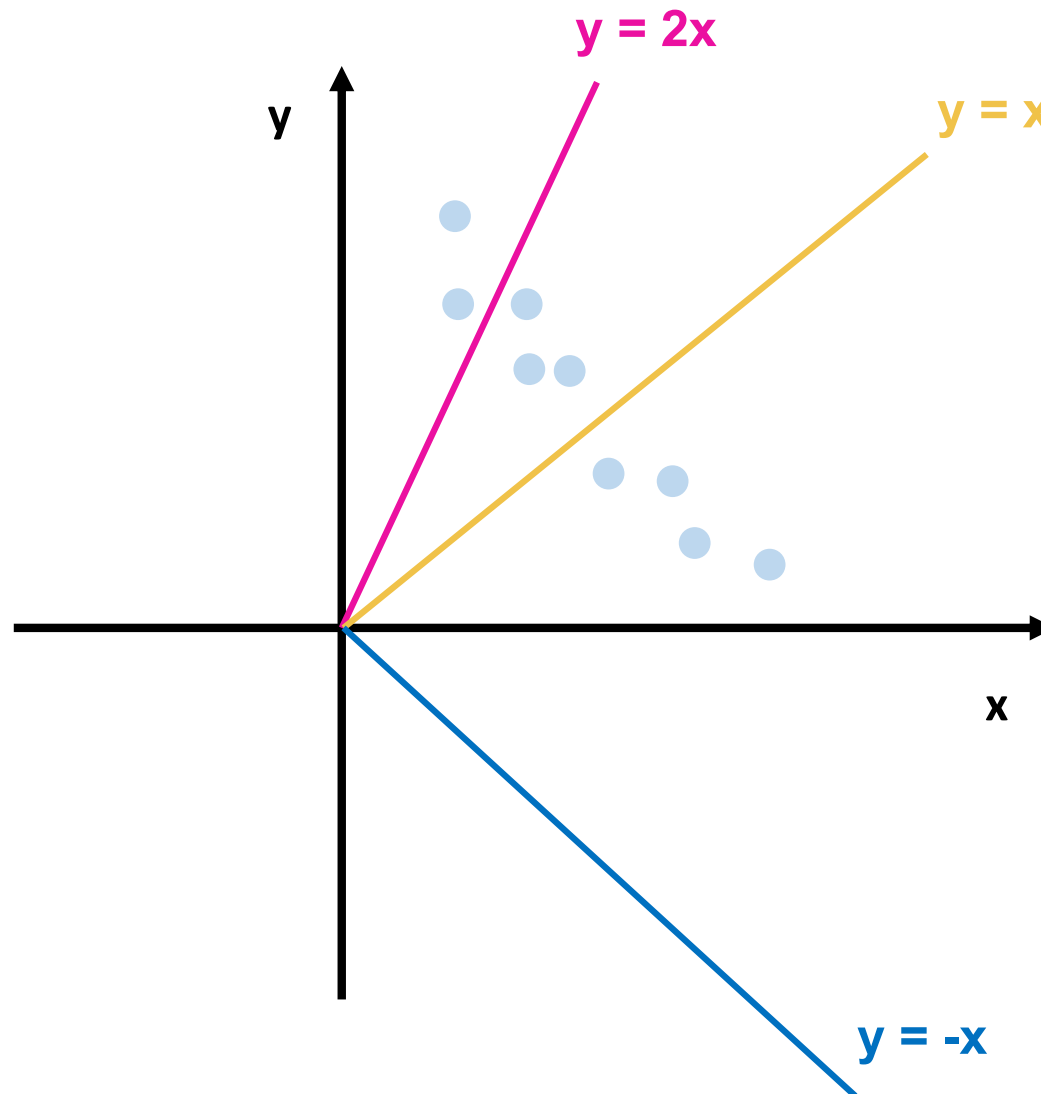
Função de ativação Step (Degrau)

$$y = \mathcal{F}(S) = \begin{cases} 1, & \text{if } S \geq 0 \\ 0, & \text{if } S < 0 \end{cases}$$



Redes Neuronais: perceptron

Bias na regressão linear



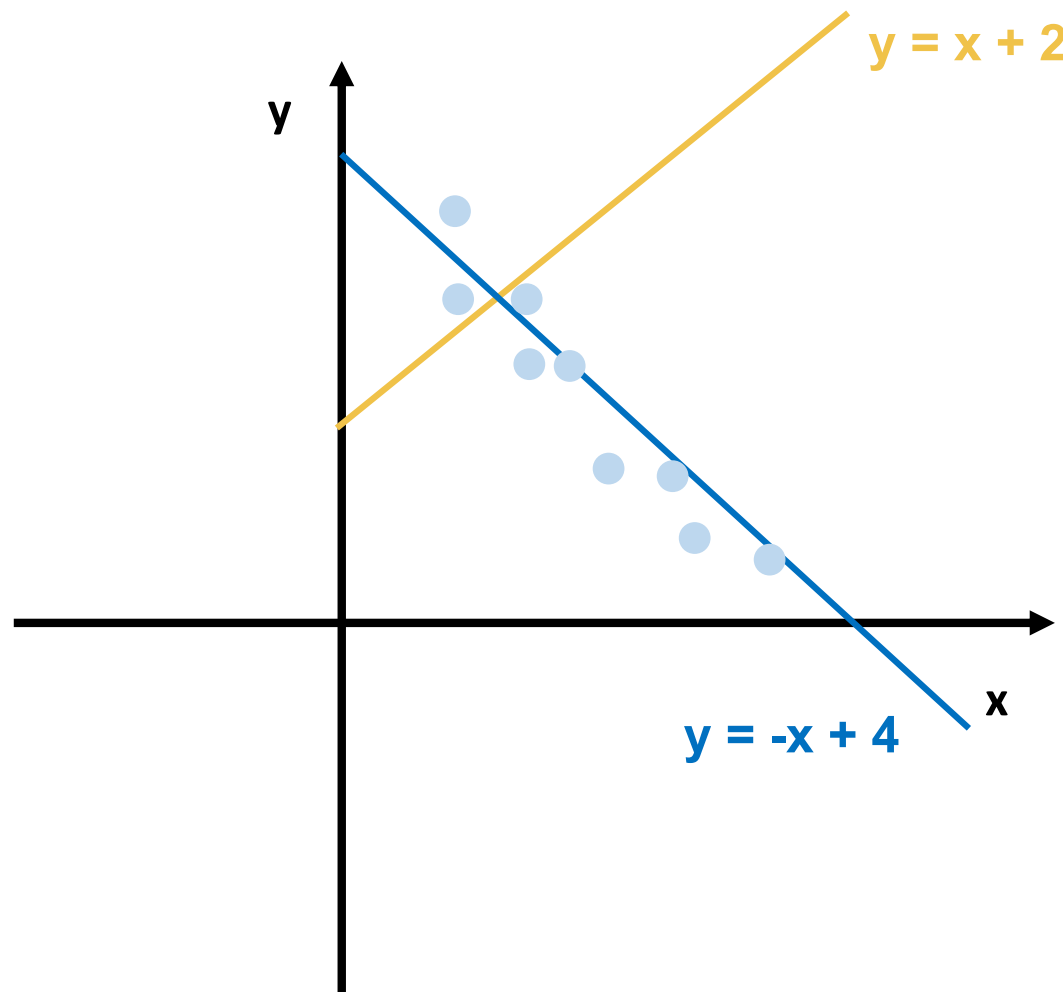
Equação da reta:

$$y = ax + b$$

↑
bias

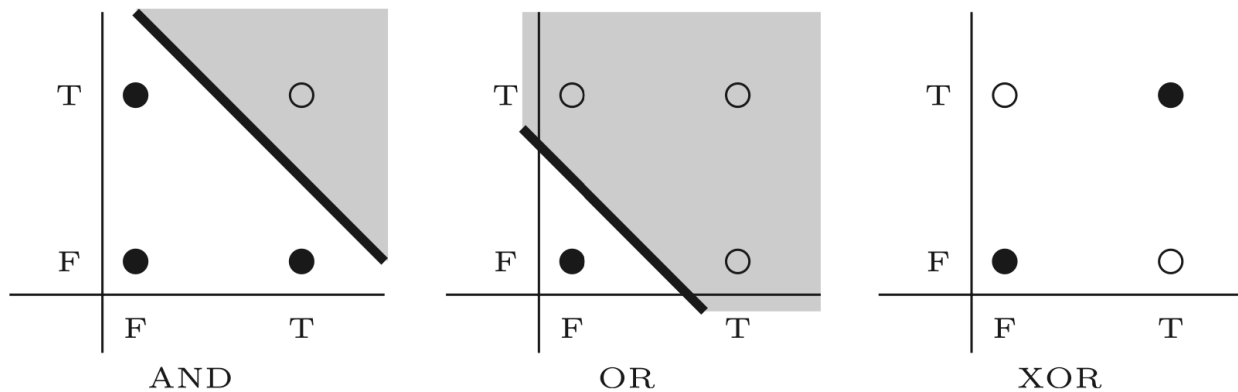
Redes Neuronais: perceptron

Bias na regressão linear

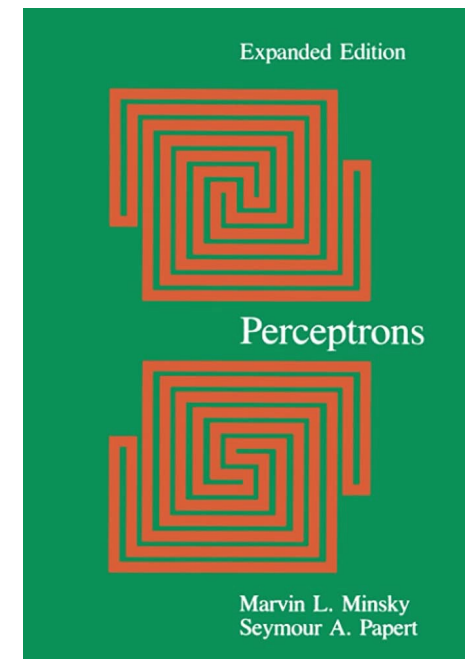


Redes Neurais: perceptron

- Limitações
 - O perceptron só aprende funções **linearmente separáveis**

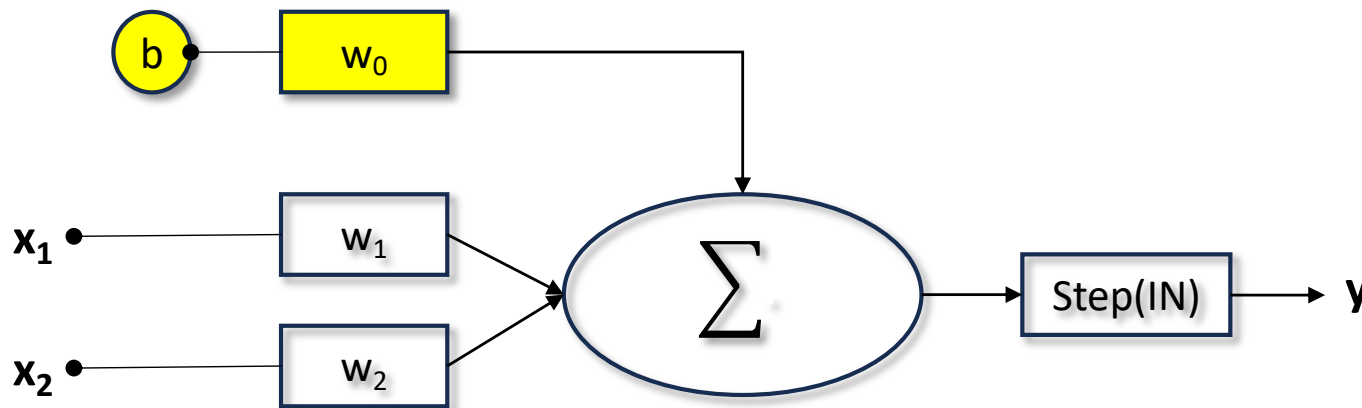


Minsky & Papert (1969)



Redes Neurais: perceptron

- Como encontrar a linha de decisão do **perceptron**?



EQUAÇÃO DA RETA

$$bw_0 + x_1w_1 + x_2w_2 = 0 \quad \Longrightarrow \quad x_2 = -x_1 \frac{w_1}{w_2} - b \frac{w_0}{w_2}$$

Redes Neurais: perceptron

- Como encontrar a linha de decisão do **perceptron**?

ex	x1	x2	t
1	0	0	0
2	0	1	1
3	1	0	1
4	1	1	1

$$b = 1$$

$$w_0 = -0.5$$

$$w_1 = 0.5$$

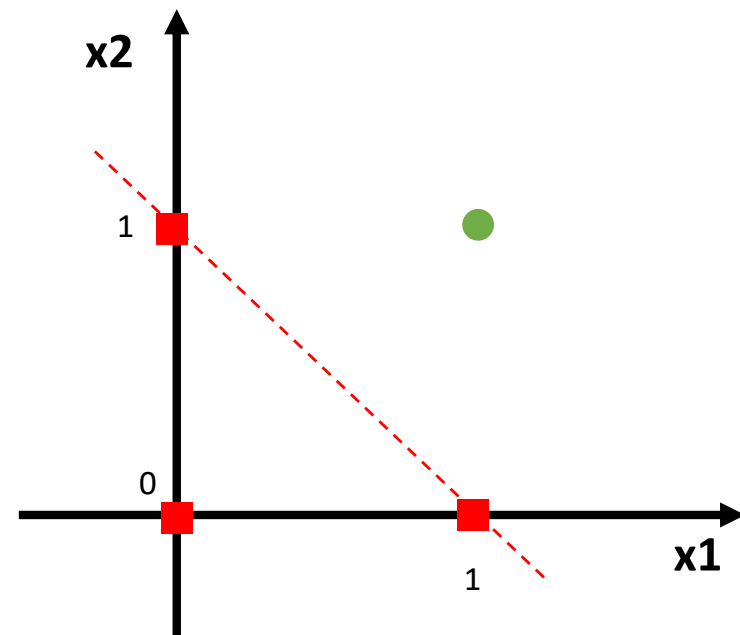
$$w_2 = 0.5$$

$$x_2 = -x_1 \frac{w_1}{w_2} - b \frac{w_0}{w_2}$$

$$x_1 = -x_2 \frac{w_2}{w_1} - b \frac{w_0}{w_1}$$

$$x_1 = 0 \rightarrow x_2 = 1$$

$$x_2 = 0 \rightarrow x_1 = 1$$

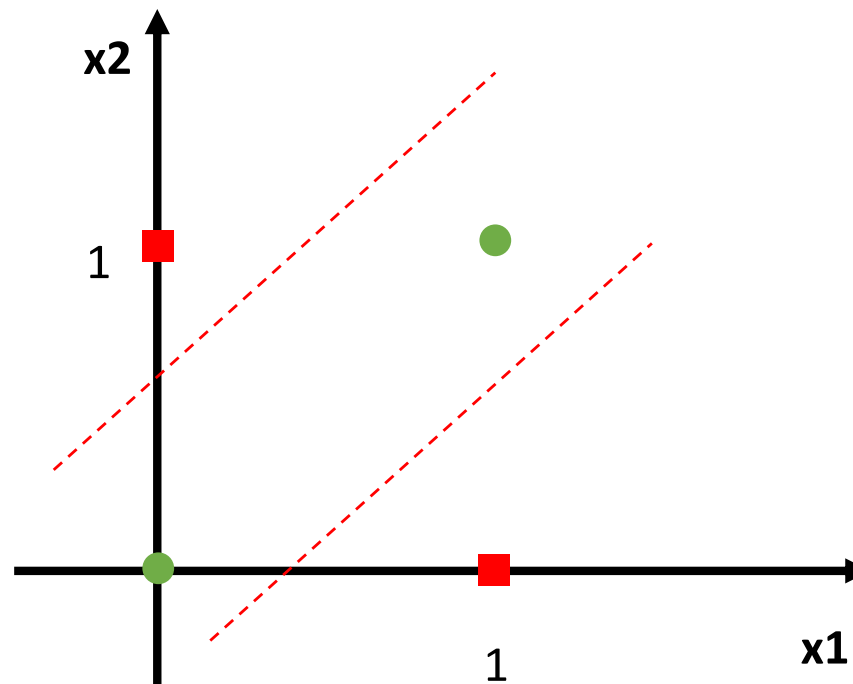


Redes Neurais: perceptron

- Limitações
 - O perceptron só aprende funções **linearmente separáveis**

XOR

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0



Redes Neurais: multi-layered perceptron

